

Object Oriented Programming In Java

Saurabh Pandey
Associate Professor
Sitare University

Stack and Heap memory areas

- Java has two areas of memory we care about: the Stack and the Heap.
- Instance variables are variables declared inside a class but outside any method.
- Local variables are variables declared inside a method or method parameter.
- All local variables live on the stack, in the frame corresponding to the method where the variables are declared.
- Object reference variables work just like primitive variables—if the reference is declared as a local variable, it goes on the stack.
- All objects live in the heap, regardless of whether the reference is a local or instance variable.

Constructors

- A constructor has the code that runs when you instantiate an object. In other words, the code that runs when you say `new` on a class type. Every class you create has a constructor, even if you don't write it yourself.
- If you have more than one constructor in a class, it means you have overloaded constructors.
- If you write a constructor that takes arguments, and you still want a no-arg constructor, you'll have to build the no-arg constructor yourself!
- If you have more than one constructor in a class, the constructors MUST have different argument lists.
- Constructors can be public, protected, private, or default (which means no access modifier at all).
- All the constructors in an object's inheritance tree must run when you make a new object.

//Show code example

- The compiler puts in a call to `super()` if you don't provide in subclass constructor.
- The call to `super()` must be the first statement in each constructor

Strings and String APIs in Java

- String basics
- String Pool
- Common APIs
- Best practices

What is a String?

- A **sequence of characters**.
- In Java: implemented as an **immutable object**.
- Stored in **String Pool** when created using string literals.

Example

`String name = "Alice"; // in String pool`

`String city = new String("Paris"); // new object in heap`

String Pool

- Special memory area inside the **Java Heap**.
- Stores **unique string literals**.
- Helps in **memory optimization**.
- Reuses already created strings instead of duplicating.

```
String a = "Java";  
String b = "Java";    // points to same object in pool  
System.out.println(a == b); // true
```

```
String c = new String("Java");  
System.out.println(a == c); // false (different object)
```

Immutability

- Strings are **immutable**.
- Any modification creates a **new object**.

Ex:

```
String s = "Hello";  
s.concat(" World");  
System.out.println(s); // "Hello"
```

Benefits: Thread-safety, caching

Drawback: Expensive for repeated modifications

String Creation

Literal (uses pool):

```
String str1 = "Java";
```

Using new (heap):

```
String str2 = new String("Java");
```

From char array:

```
char[] arr = {'J','a','v','a'};
```

```
String str3 = new String(arr);
```


Common String APIs - Inspection

```
int len = str.length();    // length  
char c = str.charAt(2);    // get char at index  
boolean empty = str.isEmpty();
```

Common String APIs - Comparison

```
str.equals("Java");  
str.equalsIgnoreCase("java");  
str.compareTo("Python");
```

Common String APIs - Modification

```
str.toUpperCase();  
str.toLowerCase();  
str.trim();  
str.replace("a", "@");  
str.substring(1, 3);
```

Common String APIs - Searching

```
str.contains("av");  
str.startsWith("Ja");  
str.endsWith("va");  
str.indexOf("a");  
str.lastIndexOf("a");
```

Splitting & Joining

```
String fruits = "apple,banana,orange";  
String[] arr = fruits.split(",");  
for(String f : arr) System.out.println(f);
```

```
String joined = String.join(" | ", "A", "B", "C");  
// A | B | C
```

String vs StringBuilder vs StringBuffer

String: Immutable.

StringBuilder: Mutable, faster, not synchronized.

StringBuffer: Mutable, synchronized, thread-safe.

```
StringBuilder sb = new StringBuilder("Hello");  
sb.append(" World");  
System.out.println(sb); // Hello World
```

Best Practices

- Use `StringBuilder` for concatenation in loops.
- Use `.equals()` not `==` for content comparison.
- Prefer literals over `new String()`.
- Use `String.join()` or `String.format()` for readability.